GAS × Claude Code 演習課題集

このドキュメントは、**Claude Code に指示を出しながら Google Apps Script（GAS）システムをゼロから実装する**ための演習課題集です。

各課題は**既存スクリプトを参考にせず、記載された仕様だけを頼りに1から実装する**ことを前提にしています。
お手本のコードはありません。仕様を読み解き、Claude Code に適切な指示を出して完成させてください。

この演習のねらい

コードを自分で手書きすることが目的ではありません。
Claude Code にどう依頼し、出てきたコードをどう確認・修正させるかを練習します。
各課題には「Claude Code 活用のポイント」を用意しているので、そこを意識して取り組んでください。

共通ルール（全課題で守ること）

1. 実装前に、仕様を自分の言葉で Claude Code に説明し、**設計方針（関数構成）を先に出力させてから**コードを書かせる。
2. CLAUDE.md のコーディング規約に従わせる（定数の集約・一括読み書き・機密情報は `PropertiesService` ・日本語コメントなど）。
3. 機密情報（API キー・Webhook URL）は**絶対にコードへ直書きしない**。スクリプトプロパティのキー名まで含めて指示する。
4. 1 課題ごとに、CLAUDE.md の Git 運用ルールに従い `git add` → `commit` → `push` まで実施する。
5. 動かないときは、エラーメッセージをそのまま Claude Code に貼り付けて原因と修正案を出させる。

共通の技術前提

- **言語 / 実行環境**: Google Apps Script（V8 ランタイム）
- **Claude API**（課題1・課題3で使用）

    - エンドポイント: `https://api.anthropic.com/v1/messages`（`UrlFetchApp.fetch` で POST）
    - ヘッダー: `x-api-key`（API キー）、`anthropic-version: 2023-06-01`、`content-type: application/json`
    - モデル: `claude-haiku-4-5`（最新の Haiku。コスト重視）
    - `muteHttpExceptions: true` を指定し、HTTP ステータスが 2xx 以外なら本文付きでエラーにする
- **スクリプトプロパティのキー名**（`PropertiesService.getScriptProperties()` から取得）
    - `CLAUDE_API_KEY`：Claude API キー
    - `SLACK_WEBHOOK_URL`：Slack Incoming Webhook の URL

事前準備（共通セットアップ）

各課題を動かす前に必要なセットアップです。課題ごとに「どれが必要か」は各課題の **事前準備** に記載します。

GAS プロジェクトの作成（全課題共通）

1. Google ドライブで**スプレッドシートを新規作成**する。
2. スプレッドシートのメニュー **拡張機能** → **Apps Script** を開く（スプレッドシートにバインドされた GAS プロジェクトが開く）。
3. `.gs` ファイルにコードを記述する。

Claude API キーの取得（課題1・課題3で必要）

1. Anthropic Console にログインする。
2. 左メニュー **API keys** → **Create Key** でキーを作成する。
    - キー名（Key Name）には用途がわかる名前を付ける。例: `gas-training`、`akita-mail-classifier`、`研修用`
3. 表示されたキー（例: `sk-ant-api03-xxxxxxxx...`）を**コピーする**（後から再表示できないため注意）。
4. GAS エディタの ⚙ **プロジェクトの設定** → **スクリプト プロパティ** → **スクリプト プロパティを追加** で登録する。
    - プロパティ名: `CLAUDE_API_KEY` ／ 値: コピーしたキー（例: `sk-ant-api03-xxxxxxxx...`）
5. ※ API の利用には Anthropic 側の**課金設定（クレジット購入）**が必要です。

Slack Incoming Webhook の設定（課題1・課題3で必要）

1. Slack API: Your Apps で **Create New App** → **From scratch** を選ぶ。
2. アプリ名と対象ワークスペースを選んで作成する。
   - アプリ名（App Name）には用途がわかる名前を付ける。例:
     応募メール通知Bot 、 **問い合わせ週次レポート**
3. 左メニュー **Incoming Webhooks** をオンにする。
4. **Add New Webhook to Workspace** を押し、投稿先チャンネルを選んで許可する。
   - 投稿先チャンネルの例: **#採用通知** 、 **#問い合わせレポート** （事前にチャンネルを作成しておく）
5. 発行された Webhook URL（例: `https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXX` ）を**コピーする**。
6. GAS のスクリプト プロパティに登録する。
   - プロパティ名: `SLACK_WEBHOOK_URL` ／ 値: コピーした URL（例: `https://hooks.slack.com/services/T0.../B0.../XXXX` ）

課題1　採用応募メールの自動仕分け・通知システム

概要

採用担当に届く応募メールを自動で分類・記録・通知するシステムを実装する。
Gmail から対象メールを取得し、Claude API で分類・情報抽出し、スプレッドシートへ記録、
Slack へ通知したうえで、処理済みメールのラベルを付け替える。

事前準備

- **スプレッドシート**: 新規作成し GAS プロジェクトをバインドする（共通セットアップ参照）。
  記録用の「応募ログ」「エラーログ」シートは**スクリプトが自動作成する**ため手動作成は不要。
- **Gmail**: 「応募」ラベルを作成し、動作確認用に未読のテストメールを数通、そのラベルに付けておく。
- **Claude API キー**: 共通セットアップの手順で `CLAUDE_API_KEY` をスクリプト プロパティに設定する。
- **Slack**: 共通セットアップの手順で `SLACK_WEBHOOK_URL` をスクリプト プロパティに設定する。

仕様

1. 対象メールの取得

- Gmail の「応募」ラベルが付いた**未読スレッド**を検索して取得する。
- 検索クエリ例: `label:"応募" is:unread`
- 1回の実行で処理する最大スレッド数は **10**（定数で管理）。
- スレッド内の**未読メッセージのみ**を処理対象とする。

2. Claude API による分類・抽出

- メール本文（プレーンテキスト）を Claude API に渡し、次の3項目を判定させる。

項目	内容
`category`（応募区分）	**`新卒`** / **`中途`** / **`アルバイト`** / **`その他`** のいずれか1つ
`position`（希望職種）	本文から読み取れる希望職種。読み取れなければ **`不明`**
`summary`（要約）	本文を日本語で1〜2文に要約

- Claude には **JSON 形式のみ**で回答させる:
  `{"category": "...", "position": "...", "summary": "..."}`
- JSON の前後に説明文が付いても解析できるよう、**最初の `{` 〜 最後の `}` を抽出**してパースする。
- `category` が上記4区分以外だった場合は **`その他`** **に寄せる**（フォールバック）。

3. スプレッドシートへの記録

- 「応募ログ」シートへ1行追記する。シートが無ければ作成し、1行目に太字のヘッダーを書き込む。
- 列構成（左から順）:

受信日時	応募者メール	件名	応募区分	希望職種	要約

4. Slack への通知

- Incoming Webhook へ、件名・応募区分・希望職種・要約を含むメッセージを POST する。

5. ラベルの付け替え

- 処理したスレッドに「選考中」ラベルを付け（無ければ作成）、「応募」ラベルを外す。
- 処理したメッセージは既読にする。

6. エラー処理

- **1通の処理に失敗しても他メールの処理を止めない。**
- 失敗したら「エラーログ」シート（列: 発生日時・件名・エラー内容）へ記録して次へ進む。

7. トリガー

- 5分おきに自動実行する時間主導型トリガーを登録する関数を用意する。
- 登録時は**既存の同名トリガーを削除してから**作成し、重複登録を防ぐ。

使う技術・GAS サービス

`GmailApp` 、 `UrlFetchApp` （Claude API / Slack）、 `SpreadsheetApp` 、 `PropertiesService` 、 `ScriptApp` （トリガー）

Claude Code 活用のポイント

- いきなり全部書かせず、まず **関数構成（取得・分類・記録・通知・ラベル操作・トリガー）を一覧で設計させる**。
- Claude API へ JSON 形式で返させるプロンプトを、Claude Code と相談しながら設計する。
- 想定外カテゴリの**フォールバック処理**を忘れず入れさせる。

完成条件（チェックリスト）

- ☐ API キー・Webhook URL がコードに直書きされていない
- ☐ 分類カテゴリ・シート名・列見出し・最大スレッド数がファイル先頭の定数にまとまっている
- ☐ Claude のレスポンスが JSON 以外を含んでもパースできる
- ☐ 1 通の処理失敗が他メールの処理を止めない（エラーログに記録して継続する）
- ☐ トリガー登録関数が重複登録を防いでいる
- ☐ 実装後にコミット＆プッシュ済み

発展（任意）

- 応募区分ごとに通知先 Slack チャンネルを変える。
- 要約だけでなく「一次返信メールの下書き」も Claude に生成させ、下書きとして Gmail に保存する。

課題2　研修アンケート集計ダッシュボード

概要

研修後アンケートの回答を自動集計し、サマリーシートとグラフで可視化するダッシュボードを実装する。

事前準備

- **スプレッドシート**: 新規作成し GAS プロジェクトをバインドする（共通セットアップ参照）。
  - 「アンケート回答」シートを作り、1行目にヘッダー（回答日時・研修名・満足度・理解度・自由記述）、
    2行目以降に動作確認用のテストデータを入力する（複数の研修名・5段階の数値を含めると集計を確認しやすい）。
  - 出力用の「研修サマリー」シートは**スクリプトが自動作成する**ため手動作成は不要。
- **Claude API / Slack**: 本課題では不要（発展課題で使う場合のみ共通セットアップを参照）。

仕様

1. 入力データ

- 入力元「アンケート回答」シートの列構成（1行目はヘッダー）：

列	内容
A	回答日時（日付）
B	研修名（文字列）
C	満足度（1〜5の数値）
D	理解度（1〜5の数値）
E	自由記述（文字列）

- データ読み込みは `getValues()` で**一括取得**する（ループ内で都度アクセスしない）。

2. 集計

- **研修名ごと**に次を集計する。
  - 回答件数
  - 満足度の平均（小数第1位まで）

    - 理解度の平均（小数第1位まで）
- 不正データはスキップする。
    - 研修名（B列）が空の行
    - 満足度・理解度（C・D列）が数値でない行

3. 出力

- 「研修サマリー」シートへ書き込む。シートが無ければ作成する。
- **書き込み前に既存の内容と既存グラフを毎回クリア**してから書き直す。
- 列構成（1行目は太字ヘッダー）：

研修名	回答件数	満足度平均	理解度平均

- 書き込みは `setValues()` で**一括書き込み**する。

4. グラフ

- 研修ごとの**平均満足度**を**棒グラフ**で可視化し、サマリーシートに挿入する。
- タイトル例: 「研修別 平均満足度」
- **データが0件のときはグラフを作成せず正常終了**する（エラーにしない）。

5. トリガー

- 毎週月曜9時台に自動実行する時間主導型トリガーを登録する関数を用意する。
- 登録時は既存の同名トリガーを削除してから作成し、重複登録を防ぐ。

使う技術・GAS サービス

`SpreadsheetApp` 、 `Charts` （グラフ）、 `ScriptApp` （トリガー）

Claude Code 活用のポイント

- 「読み込み → 集計 → 書き込み → グラフ」の**関数構成を実装前に説明させる**。
- 不正データ（数値でない・空欄）をスキップする処理を必ず入れさせる。
- `getValues()` / `setValues()` による**一括読み書き**になっているか、生成コードをレビューさせる。

完成条件（チェックリスト）

- [ ] シート名・列インデックスが先頭の定数にまとまっている
- [ ] データ0件でもエラーにならない（グラフは作らず正常終了する）

- [ ] 平均値が小数第1位で表示される
- [ ] ループ内でシートへ都度アクセスしていない（一括処理になっている）
- [ ] 書き込み前に既存グラフをクリアしている（グラフが重複しない）
- [ ] 実装後にコミット＆プッシュ済み

発展（任意）

- 満足度の平均が基準値（例：3.5）を下回った研修があれば、Slack に警告通知する。
- 自由記述（E列）を Claude API に渡し、研修ごとの「よくある意見」を要約させて併記する。

課題3　問い合わせ対応 週次レポート自動配信

概要

蓄積された問い合わせログを週次で集計し、集計結果を Claude API に渡して
日本語のサマリー文章を生成させ、Slack へ自動配信するシステムを実装する。

事前準備

- **スプレッドシート**: 新規作成し GAS プロジェクトをバインドする（共通セットアップ参照）。
  - 「メールログ」シートを作り、1行目にヘッダー（受信日時・送信者・件名・分類・要約）、2行目以降にテストデータを入力する。動作確認のため、**受信日時が直近7日以内の行を必ず含める**こと。
  - （課題1を実装済みであれば、その出力「応募ログ／メールログ」を流用してもよい）
- **Claude API キー**: 共通セットアップの手順で `CLAUDE_API_KEY` をスクリプト プロパティに設定する。
- **Slack**: 共通セットアップの手順で `SLACK_WEBHOOK_URL` をスクリプト プロパティに設定する。

仕様

1. 入力データ

- 入力元「メールログ」シートの列構成（1行目はヘッダー）：

列	内容
A	受信日時（日付）

列	内容
B	送信者
C	件名
D	分類（ **クレーム** / **質問** / **注文** / **その他** ）
E	要約

2. 集計

- **直近7日間**（実行時点から7日前以降）の行だけを対象にする。
  - 受信日時（A列）が日付として読めない行はスキップする。
  - 7日より古い行はスキップする。
  - 日付比較は**境界値・タイムゾーン**に注意する。
- 集計する値:
  - 総件数
  - 分類（クレーム / 質問 / 注文 / その他）ごとの件数
  - 想定外の分類は **その他** に寄せる。

3. Claude API による文章生成

- **集計済みの数値（総件数・分類別件数）だけ**を Claude API に渡す。
- **個別メールの本文・要約・送信者は渡さない**（トークン節約とプライバシー配慮のため）。
- 社内向けの週次サマリーを**日本語3〜4文の文章**（箇条書きでなく文章）で生成させる。

4. Slack への配信

- Claude が生成した文章と、分類別内訳（総件数含む）を Slack へ POST する。
- **対象ログが0件のときもエラーにせず**、「今週の問い合わせはありませんでした」の旨を通知する。

5. トリガー

- 毎週金曜17時台に自動実行する時間主導型トリガーを登録する関数を用意する。
- 登録時は既存の同名トリガーを削除してから作成し、重複登録を防ぐ。

使う技術・GAS サービス

`SpreadsheetApp` 、 `UrlFetchApp` （Claude API / Slack）、 `PropertiesService` 、 `ScriptApp` （トリガー）、日付計算

Claude Code 活用のポイント

- 「集計 → Claude で文章化 → Slack 配信」の**処理を関数に分割する設計**を、実装前に箇条書きで出力させる。
- 「直近7日間」の抽出で日付比較が正しいか（タイムゾーン・境界値）を、Claude Code に確認させる。
- Claude へ渡すのは**集計値のみ**でログ全文は渡さない設計にする。この狙いを自分の言葉で Claude Code に伝えて実装させる。

完成条件（チェックリスト）

- ☐ 対象期間（7日間）の抽出が正しく動作する
- ☐ ログが0件のときもエラーにならず、その旨を通知する
- ☐ Claude への入力に個別メール本文を含めていない（集計値のみ）
- ☐ API キー・Webhook URL がスクリプトプロパティ管理になっている
- ☐ トリガー登録関数が重複登録を防いでいる
- ☐ 実装後にコミット＆プッシュ済み

発展（任意）

- レポートをテキストだけでなく、集計結果をグラフ化して添えて配信する。
- 月初には「前月まとめレポート」を別途生成する関数を追加する。

取り組み方の目安

課題	テーマ	難易度	主な練習ポイント
課題1	採用応募メールの自動仕分け	★★☆	関数構成の設計・API プロンプト設計・フォールバック
課題2	研修アンケート集計ダッシュボード	★★☆	集計ロジック・一括処理・グラフ・ 0件対応
課題3	問い合わせ週次レポート配信	★★★	処理分割の設計・複数サービス連携・ プライバシー配慮

各課題の最後には、**「Claude Code にどう指示したら、どんなコードが返ってきたか」を振り返り**、うまくいった指示の出し方をメモしておきましょう。

解答例（リファレンス実装）

各課題の解答例を用意しています。**まずは自力で Claude Code に実装させてから**、
答え合わせや別解の確認として参照してください。

課題	解答例ファイル	主な関数
課題1	solution_recruit_classifier.gs	`classifyRecruitEmails` / `setRecruitClassifyTrigger`
課題2	solution_survey_dashboard.gs	`runSurveyDashboard` / `setSurveyWeeklyTrigger`
課題3	solution_weekly_report.gs	`sendWeeklyReport` / `setWeeklyReportTrigger`

解答例を使うときの注意

- GAS は1プロジェクト内の全 `.gs` ファイルが**同じグローバルスコープ**を共有します。
  解答例どうし・既存ファイルとの定数名の衝突を避けるため、各解答例の定数には接頭辞（課題1= `RC_` 、課題2= `SV_` 、課題3= `WR_` ）を付けています。
- 実際に動かす際は、**解答例ごとに別々の GAS プロジェクト**へ貼り付けて使うことを想定しています。
- 課題1・課題3は、実行前にスクリプトプロパティ（ `CLAUDE_API_KEY` / `SLACK_WEBHOOK_URL` ）の設定が必要です。
- いずれも Claude のモデルは `claude-haiku-4-5` （最新の Haiku）を使用しています。

解答例プロンプト（Claude Code への指示例）

上記の解答例を導くための **Claude Code への指示プロンプト例**です。
そのままコピーして使うのではなく、「どういう粒度・順序で依頼すると意図どおりのコードが返るか」を
読み取る教材として活用してください。**いきなり全部書かせず、まず設計を出させて合意してから実装させる**のがコツです。

課題1 解答例プロンプト（採用応募メール分類）

> GAS で、採用応募メールを自動で仕分けするスクリプトを `solution_recruit_classifier.gs` という
> 新規ファイルで1から作りたいです。いきなりコードは書かず、**まず関数構成（取得・分類・記録・通知・**
> **ラベル操作・トリガー）を箇条書きで設計**してください。
>
> 仕様は次のとおりです。
>
> - Gmail の「応募」ラベルが付いた未読メールを取得する（検索クエリ `label:"応募" is:unread` 、1回の実行で最大10スレッド、未読メッセージのみ処理）。
> - 本文を Claude API に渡し、 **応募区分** （新卒/中途/アルバイト/その他）・ **希望職種** （不明可）・ **要約** （1〜2文）を JSON `{"category":..,"position":..,"summary":..}` 形式のみで返させる。JSON 前後に文字が付いても `{` 〜 `}` を抽出してパースし、想定外の区分は「その他」に寄せる。
> - 結果を「応募ログ」シート（受信日時・応募者メール・件名・応募区分・希望職種・要約）へ追記する。シートが無ければ太字ヘッダー付きで作成する。
> - Slack の Incoming Webhook へ件名・応募区分・希望職種・要約を通知する。
> - 処理済みメールに「選考中」ラベルを付け（無ければ作成）「応募」ラベルを外し、メッセージを既読にする。
> - 5分おきに実行するトリガー登録関数も作る（重複登録を防ぐこと）。
>
> 技術前提: Claude API は `https://api.anthropic.com/v1/messages` へ POST、ヘッダーは `x-api-key` と `anthropic-version: 2023-06-01` 、モデルは `claude-haiku-4-5` 、 `muteHttpExceptions: true` で 2xx 以外はエラーにする。
> API キーと Webhook URL はスクリプトプロパティ（ `CLAUDE_API_KEY` / `SLACK_WEBHOOK_URL` ）から取得し、コードに直書きしない。
> 1通の処理に失敗しても他メールの処理を止めず、エラーは「エラーログ」シート（発生日時・件名・エラー内容）へ記録して継続する。
> `CLAUDE.md` の規約に従い、定数は先頭にまとめ、他ファイルと衝突しないよう接頭辞 `RC_` を付けてください。
>
> 設計の合意が取れたら実装し、最後に「エラー継続・フォールバック・直書き無し」の観点で自己レビューしてください。

課題2 解答例プロンプト（研修アンケート集計ダッシュボード）

> GAS で、研修アンケートを集計するダッシュボードを `solution_survey_dashboard.gs` という新規ファイルで1から作りたいです。**まず「読み込み → 集計 → 書き込み → グラフ」の関数構成を
> 箇条書きで説明**してから実装してください。
>
> 仕様は次のとおりです。
>
> - 入力元「アンケート回答」シートの列: A=回答日時 / B=研修名 / C=満足度(1〜5) / D=理解度(1〜5) / E=自由記述。
> - `getValues()` で一括取得し、**研修名ごと**に「回答件数」「満足度平均」「理解度平均」（小数第1位）を集計する。
> - 研修名が空の行、満足度・理解度が数値でない行はスキップする。
> - 「研修サマリー」シート（無ければ作成）へ、既存の内容と既存グラフを毎回クリアしてから
>   「研修名・回答件数・満足度平均・理解度平均」を太字ヘッダー付きで `setValues()` 一括書き込みする。
> - 研修別の平均満足度を棒グラフ（タイトル「研修別 平均満足度」）で可視化し挿入する。データ0件ならグラフを作らず正常終了する。
> - 毎週月曜9時に実行するトリガー登録関数も作る（重複登録を防ぐこと）。
>
> `CLAUDE.md` の規約に従い、シート名・列インデックスは先頭の定数にまとめ、接頭辞 `SV_` を付けてください。
>
> 実装後、**「ループ内でシートに都度アクセスしていないか」「0件で落ちないか」「グラフが重複しないか」**を自己レビューしてください。

課題3 解答例プロンプト（週次レポート自動配信）

> GAS で、問い合わせ対応の週次レポートを Slack へ自動配信するスクリプトを `solution_weekly_report.gs` という新規ファイルで1から作りたいです。**まず「集計 → Claude で文章化 →
> Slack 配信」を関数に分割した設計**を示してから実装してください。
>
> 仕様は次のとおりです。
>
> - 入力元「メールログ」シートの列: A=受信日時 / B=送信者 / C=件名 / D=分類（クレーム/質問/注文/その他） / E=要約。

- 直近7日間（実行時点から7日前以降）の行を対象に、総件数と分類別件数を集計する。受信日時が日付でない行や7日より古い行はスキップし、想定外の分類は「その他」に寄せる。日付比較は境界値・タイムゾーンに注意する。
- **集計値（総件数・分類別件数）だけ**を Claude API に渡し、社内向けの週次サマリーを日本語3〜4文の文章で生成させる。
  個別メールの本文・要約・送信者は Claude に渡さないこと（トークン節約とプライバシー配慮のため）。
- 生成文章と分類別内訳（総件数含む）を Slack へ投稿する。ログが0件のときもエラーにせず、その旨を通知する。
- 毎週金曜17時に実行するトリガー登録関数を作る（重複登録を防ぐこと）。

技術前提: Claude API は `https://api.anthropic.com/v1/messages` へ POST、モデルは `claude-haiku-4-5` 、
`muteHttpExceptions: true` で 2xx 以外はエラーにする。API キー・Webhook URL はスクリプトプロパティから取得する。
`CLAUDE.md` の規約に従い、定数は先頭にまとめ、接頭辞 `WR_` を付けてください。

実装後、**「Claude への入力に本文が混ざっていないか」「7日間の抽出が正しいか」**を重点的に自己レビューしてください。